

Mockito Hands-On Exercises

Step 1: Create ExternalApi.java

```
public interface ExternalApi {  
  
    String getData();  
  
    void saveData(String data);  
  
    void deleteData();  
  
    void process(String name, int age);  
}
```

Step 2: Create MyService.java

```
public class MyService {  
  
    private ExternalApi api;  
  
    public MyService(ExternalApi api) {  
        this.api = api;  
    }  
  
    public String fetchData() {  
        return api.getData();  
    }  
  
    public void save(String data) {  
        api.saveData(data);  
    }  
  
    public void delete() {  
        api.deleteData();  
    }  
  
    public void processUser(String name, int age) {  
        api.process(name, age);  
    }  
}
```

Exercise 1: Mocking and Stubbing

Create MockingTest.java

```
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class MockingTest {

    @Test
    public void testExternalApi() {

        ExternalApi mockApi = mock(ExternalApi.class);

        when(mockApi.getData())
            .thenReturn("Mock Data");

        MyService service = new MyService(mockApi);

        String result = service.fetchData();

        assertEquals("Mock Data", result);
    }
}
```

Exercise 2: Verifying Interactions

Create VerifyInteractionTest.java

```
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class VerifyInteractionTest {

    @Test
    public void testVerifyInteraction() {

        ExternalApi mockApi = mock(ExternalApi.class);

        MyService service = new MyService(mockApi);
```

```
        service.fetchData();

        verify(mockApi).getData();
    }
}
```

Exercise 3: Argument Matching

Create ArgumentMatcherTest.java

```
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class ArgumentMatcherTest {

    @Test
    public void testArgumentMatching() {

        ExternalApi mockApi = mock(ExternalApi.class);

        MyService service = new MyService(mockApi);

        service.processUser("John", 25);

        verify(mockApi)
            .process(eq("John"), anyInt());
    }
}
```

Exercise 4: Handling Void Methods

Create VoidMethodTest.java

```
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class VoidMethodTest {

    @Test
    public void testVoidMethod() {
```

```
ExternalApi mockApi = mock(ExternalApi.class);

doNothing()
    .when(mockApi)
    .saveData(anyString());

MyService service = new MyService(mockApi);

service.save("Hello");

verify(mockApi).saveData("Hello");
}
}
```

Exercise 5: Multiple Returns

Create MultipleReturnsTest.java

```
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class MultipleReturnsTest {

    @Test
    public void testMultipleReturns() {

        ExternalApi mockApi = mock(ExternalApi.class);

        when(mockApi.getData())
            .thenReturn("First")
            .thenReturn("Second")
            .thenReturn("Third");

        assertEquals("First", mockApi.getData());
        assertEquals("Second", mockApi.getData());
        assertEquals("Third", mockApi.getData());
    }
}
```

Exercise 6: Verify Interaction Order

Create OrderVerificationTest.java

```
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;
import org.mockito.InOrder;

public class OrderVerificationTest {

    @Test
    public void testOrder() {

        ExternalApi mockApi = mock(ExternalApi.class);

        mockApi.getData();
        mockApi.saveData("ABC");

        InOrder order = inOrder(mockApi);

        order.verify(mockApi).getData();
        order.verify(mockApi).saveData("ABC");
    }
}
```

Exercise 7: Void Method with Exception

Create VoidExceptionTest.java

```
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class VoidExceptionTest {

    @Test
    public void testVoidException() {

        ExternalApi mockApi = mock(ExternalApi.class);

        doThrow(new RuntimeException("Error"))
            .when(mockApi)
            .deleteData();
    }
}
```

```

    assertThrows(
        RuntimeException.class,
        () -> mockApi.deleteData()
    );

    verify(mockApi).deleteData();
}
}

```











